BookLeaf Author Support Portal

Comprehensive REST API Specification & System Integration Manual

Version: 1.0.0	**API Protocol:** HTTP/1.1 & HTTP/2 over TLS
Local Base URL: http://localhost:8000	**OpenAPI Docs:** http://localhost:8000/docs
Production Base URL: https://bookleaf-api.onrender.com	**OpenAPI Spec:** /openapi.json

1. Authentication & Security Architecture

All secured endpoints enforce Bearer JWT authentication via the HTTP standard Authorization: Bearer <token> header. Tokens are cryptographically signed using HMAC-SHA256 (HS256) with a 24-hour expiration window.

Role-Based Access Control (RBAC):

- **Author Scope (/api/v1/me/*):** Requires role == 'author'. Staff tokens receive 403 Forbidden.
- **Admin Scope (/api/v1/admin/*):** Requires role == 'admin'. Author tokens receive 403 Forbidden.
- **Zero-Knowledge Multi-Tenancy:** When an author queries a ticket belonging to another author, the API returns 404 Not Found (never 403 Forbidden), completely neutralizing ID enumeration and timing attacks.
- **Strict Payload Hardening:** Pydantic v2 schemas reject any untyped field injections (extra='forbid').

2. Complete Endpoints Summary Matrix

Method	Endpoint Path	Role Required	Description / Action
GET	/health	Public	Service liveness and uptime probe
GET	/health/ready	Public	Neon PostgreSQL connection pool ping
POST	/api/v1/auth/login	Public	User credential authentication (returns JWT)
GET	/api/v1/auth/me	Auth (Any)	Returns active user claims & profile
GET	/api/v1/me/books	Author	Author books, platform links, royalty balances
GET	/api/v1/me/tickets	Author	List author's own tickets with ?status= filter
POST	/api/v1/me/tickets	Author	File support inquiry (enforces book ownership)
GET	/api/v1/me/tickets/{id}	Author	Ticket thread & messages (404 for unowned)
POST	/api/v1/me/tickets/{id}/messages	Author	Post author follow-up reply
GET	/api/v1/admin/stats	Admin	Operational triage KPI counters
GET	/api/v1/admin/tickets	Admin	Global queue with multi-facet filters
GET	/api/v1/admin/tickets/{id}	Admin	Ticket inspection with internal notes & AI draft
PATCH	/api/v1/admin/tickets/{id}	Admin	Update lifecycle status or reassign staff
POST	/api/v1/admin/tickets/{id}/messages	Admin	Post official reply or private internal note
POST	/api/v1/admin/tickets/{id}/draft	Admin	Trigger Bedrock AI draft generation
GET	/api/v1/me/stream	Author	Real-time Server-Sent Events notification stream
GET	/api/v1/admin/stream	Admin	Admin queue Server-Sent Events stream

3. Detailed Endpoint Specifications

3.1 User Authentication (`POST /api/v1/auth/login`)

Authenticates author or staff credentials using case-insensitive email matching and Argon2id verification.

Request Example:

```
POST /api/v1/auth/login Content-Type: application/json { "email": "priya.sharma@email.com", "password": "demo" }
```

Response Example (200 OK):

```
{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "token_type": "bearer", "expires_in": 86400, "user": { "id": "AUTH001", "name": "Priya Sharma", "email": "priya.sharma@email.com", "role": "author", "author_id": "AUTH001" } }
```

3.2 Author Books & Royalty Ledgers (`GET /api/v1/me/books`)

Returns author's titles with platform distribution links and audited financial figures. Guarantees total_royalty_earned - royalty_paid = royalty_pending.

Response Example (200 OK):

```
[ { "id": "BK001", "title": "Whispers of the Ganges", "status": "Published & Live", "copies_sold": 450, "total_royalty_earned": 12500.00, "royalty_paid": 10000.00, "royalty_pending": 2500.00, "platforms": [ {"platform_name": "Amazon India", "url": "https://amazon.in/dp/B08XYZ101", "is_active": true}, {"platform_name": "Flipkart", "url": "https://flipkart.com/book/bk001", "is_active": true} ] } ]
```

3.3 Create Support Ticket (`POST /api/v1/me/tickets`)

Submits a new ticket. Enforces author ownership of book_id at the PostgreSQL engine level via composite foreign key tickets_book_owner_fk.

Request Example:

```
POST /api/v1/me/tickets Authorization: Bearer Content-Type: application/json { "subject": "Q3 royalty calculation discrepancy inquiry", "description": "Hello, I am checking why the Q3 disbursement does not match portal ledger sales.", "book_id": "BK001" }
```

Response Example (201 Created):

```
{ "id": "BL-1130", "subject": "Q3 royalty calculation discrepancy inquiry", "status": "Open", "priority": "Medium", "category": "royalty_inquiry", "book_id": "BK001", "book_title": "Whispers of the Ganges", "created_at": "2026-10-26T00:20:00Z", "messages": [ { "id": "msg_BL-1130_1", "sender_name": "Priya Sharma", "sender_role": "author", "body": "Hello, I am checking why the Q3 disbursement does not match portal ledger sales.", "created_at": "2026-10-26T00:20:00Z" } ] }
```

3.4 Admin Ticket Lifecycle (`PATCH /api/v1/admin/tickets/{id}`)

Allows staff to update ticket lifecycle status, urgency priority, or reassign operational agents.

Request Example:

```
PATCH /api/v1/admin/tickets/BL-1001 Authorization: Bearer Content-Type: application/json { "status": "Resolved", "assignee": "Support Lead", "priority": "High" }
```

3.5 Staff Replies & Private Notes (`POST /api/v1/admin/tickets/{id}/messages`)

Staff can reply publicly or log internal notes. Posting a public reply to an Open ticket automatically transitions its status to In progress, reducing the needs_first_reply KPI counter. Internal notes are masked from the author portal and never update the public activity timestamp.

Request Example (Private Staff Note):

```
POST /api/v1/admin/tickets/BL-1001/messages Authorization: Bearer Content-Type: application/json { "body": "Verified NEFT transaction batch pending finance signoff.", "is_internal_note": true }
```

3.6 AI Response Assistant (`POST /api/v1/admin/tickets/{id}/draft`)

Calls AWS Bedrock Mantle (minimax.minimax-m2.5) on demand. Evaluates the author's ledger facts, runs numeric fact-checking guards against the ■1,000 policy, and formats paragraphs cleanly.

Response Example (200 OK):

```
{ "ticket_id": "BL-1001", "draft": "Dear Priya,\n\nThank you for reaching out regarding your royalties for 'Whispers of the Ganges'...\n\nWarm regards,\nBookLeaf Support Team", "source": "bedrock:minimax.minimax-m2.5", "unverified_figures": false, "confidence": 0.94 }
```

4. Real-Time Server-Sent Events (SSE) Stream

Clients subscribe to real-time events over unidirectional HTTP streams at GET /api/v1/me/stream or GET /api/v1/admin/stream. Events are emitted instantly when tickets are filed, updated, or replied to.

Sample Event Stream:

```
event: ticket.message_added data: {"ticket_id": "BL-1001", "sender_role": "support", "timestamp": "2026-10-26T00:30:00Z"} event: ticket.status_changed data: {"ticket_id": "BL-1001", "old_status": "Open", "new_status": "In progress"}
```

5. RFC 7807 Standard Error Dictionary

All error responses follow the standard problem details schema across HTTP 400, 401, 403, 404, 422, and 500 status codes:

```
{ "type": "https://bookleaf.pub/errors/validation_error", "title": "Unprocessable Entity", "status": 422, "detail": "Description must be at least 10 characters.", "instance": "/api/v1/me/tickets" }
```